SV Randomization & Functional Coverage Assignment - Extra

Q1.

```
module queues;
  int j;
  int q[$] = '{0,2,5};
  initial begin
    j = 1; // j is initialized with 1
    q.insert(1,j); // insert value of j on index no. 1
    $display(q); // display elements of queue
    q.delete(1); // delete element from index no. 1
    $display(q); // display elements of queue
    q.push_front(7); // push element 7 at front of queue
    $display(q); // display elements of queue
    q.push_back(9); // push 9 at back of queue
    $display(q); // display elements of queue
    j = q.pop_back(); // pop element from back of queue
    $display(j,",",q); // display elements of queue
    j = q.pop_front(); // pop  element from front of queue
    $display(j,",",q); // display elelments of queue
    q.reverse(); // reversing of queue
    $display(q); // display elelments of queue
    q.sort(); // sorting of queue
    $display(q); // display elelments of queue
    q.rsort(); // reverse sorting of queue
    $display(q); // display elements of queue
    q.shuffle(); // shuffling of elements of queue
    $display(q); // display elements of queue
  end
endmodule
```

Kareem Ali

Q2.

```
package ADDER_LIMITS;
parameter MAXNEG = -8;
parameter MAXPOS = 7;
typedef enum {MAXNEG_ = MAXNEG, ZERO_ = 0, MAXPOS_ = MAXPOS} reg_e;

class adder_inputs;
  rand reg signed [3:0] A, B;
  rand reg signed [3:0] A_rem_values, B_rem_values;
  rand bit reset;
  rand reg_e enum_ext_A, enum_ext_B;

  constraint rst_con {
    reset dist {1:= 2, 0:= 98};
  }

  constraint A_B_con {
    A_rem_values != (MAXPOS || 0 || MAXNEG);
    B_rem_values != (MAXPOS || 0 || MAXNEG);

    A dist {enum_ext_A:= 80, A_rem_values:= 20};
    B dist {enum_ext_B:= 80, B_rem_values:= 20};
  }

  covergroup Covgrp_A ;
    A_cvp_values : coverpoint A iff (!reset) {
      bins A_data_0 = {0};
      bins A_data_max = {MAXPOS};
      bins A_data_min = {MAXNEG};
      bins A_data_default = default;
    }
    A_cvp_transitions : coverpoint A iff (!reset) {
      bins A_data_0max = (0 => MAXPOS);
      bins A_data_minmax = (MAXNEG => MAXPOS);
      bins A_data_maxmin = (MAXPOS => MAXNEG);
    }
  endgroup

  covergroup Covgrp_B ;
    B_cvp_values : coverpoint B iff (!reset) {
      bins B_data_0 = {0};
      bins B_data_max = {MAXPOS};
      bins B_data_min = {MAXNEG};
      bins B_data_default = default;
    }
    B_cvp_transitions : coverpoint B iff (!reset) {
      bins B_data_0max = (0 => MAXPOS);
      bins B_data_minmax = (MAXNEG => MAXPOS);
      bins B_data_maxmin = (MAXPOS => MAXNEG);
    }
  endgroup
  function new();
    Covgrp_A = new();
    Covgrp_B = new();
  endfunction
endclass
endpackage
```

Karnasem

```
import ADDER_LIMITS::*;
module adder_tb;
   // inputs to instantiated Adder
   reg clk, reset;
   reg  signed [3:0] A, B; // Input data A, B: 2's complement

   // Output of Adder
   wire signed [4:0] C; // Adder output: 2's complement

   integer error_count, correct_count; // 32-bit signed

   adder_inputs adder_obj = new();

   adder a1 (
         .clk(clk),
         .reset(reset),
         .A(A),    // Input data A: signed
         .B(B),    // Input data B: signed
         .C(C)        // Adder output: signed
         );

   // Create the clock
   initial begin
      clk= 0;
      forever
         #1 clk= ~clk;
   end

   initial begin
      error_count = 0;
      correct_count = 0;
      A = 0;
      B = 0;

      assert_reset;

      // Testing adder
      repeat (10000) begin
         assert(adder_obj.randomize());
         A =  adder_obj.A;
         B = adder_obj.B;
         reset = adder_obj.reset;
         adder_obj.Covgrp_A.sample();
         adder_obj.Covgrp_B.sample();
         if (reset)
            check_result(0);
         else
            check_result(A + B);
      end

      $display("%t: At end of test error count is %0d and correc
      $stop;
   end

   task check_result (input signed [4:0] expected_result);
      @(negedge clk);
      if (expected_result !== C) begin
        error_count = error_count + 1;
        $display("%t: Error: For A= 0d%0d, and B=0d%0d C should
      end
      else
        correct_count = correct_count + 1;
   endtask

   task assert_reset;
      //assert reset
      reset = 1;
      check_result(0);
      //desert reset
      reset = 0;
   endtask
endmodule
```

Q3.

```
package FSM_pkg;
typedef enum {IDLE, ZERO, ONE, STORE} state_e;

class fsm_transaction;
rand bit x, rst;
bit y_exp;
bit [9:0] users_count_exp;

constraint rst_con {
 rst dist {1:= 2, 0:= 98};
}

constraint x_con {
 x dist {0:= 67, 1:= 33};
}

covergroup cvr_grp;
    opcode_cvp_values : coverpoint x {
        bins x_trans = (0 => 1 => 0);
    }
endgroup

function new();
    cvr_grp = new();
endfunction

endclass
endpackage
```

```
import FSM_pkg::*;
module FSM_010_tb;

logic x, y;
logic [9:0] users_count;
logic clk, rst;
fsm_transaction f_txn = new;

int error_count;
int correct_count;

FSM_010 f1(clk, rst, x, y, users_count);

// Create the clock
initial begin
   clk = 0;
   forever #1 clk = !clk;
end

// Stimulus Generation
initial begin
  error_count = 0;
  correct_count = 0;
  x = 0;

  assert_reset;

  repeat(20000) begin
    assert(f_txn.randomize());
    x = f_txn.x;
    rst = f_txn.rst;
    f_txn.cvr_grp.sample();
    check_result(f_txn);
  end

  // disable reset for user_count coverage
   repeat(10000) begin
     assert(f_txn.randomize());
     x = f_txn.x;
     rst = 0;
     f_txn.rst = 0;
     f_txn.cvr_grp.sample();
     check_result(f_txn);
  end

  $display("%t: At end of test error count is %0d a
  $stop;
end
```

```
// Checker
task check_result(input fsm_transaction f_txn_c);
   golden_model(f_txn_c);
   if (f_txn_c.rst !== 1) begin
      @(negedge clk);
      if ( (f_txn_c.y_exp !== y) || (f_txn_c.users_count_exp !== users_count) ) begin
        error_count = error_count + 1;
        $display("%t: Error: For x= 0d%0d, rst=0d%0d, y should equal 0d%0d but is 0d%0d and use
      end
      else
        correct_count = correct_count + 1;
   end
   else begin //rst check
      check_rst;
   end

endtask

// Golden Model
task golden_model(input fsm_transaction f_txn_g);
   state_e cs, ns;
   case (cs)
      IDLE: begin
         if (f_txn_g.x)
            ns = IDLE;
         else
            ns = ZERO;
      end

      ZERO: begin
         if (f_txn_g.x)
            ns = ONE;
         else
            ns = ZERO;
      end

      ONE: begin
         if (f_txn_g.x)
            ns = IDLE;
         else
            ns = STORE;
      end
```

Kareem Waseem

```
      STORE: begin
         if (f_txn_g.x)
            ns = IDLE;
         else
            ns = ZERO;
      end
   endcase
   @(posedge clk);
   if (f_txn_g.rst) begin
      cs <= IDLE;
      f_txn_g.users_count_exp <= 0;
   end
   else begin
      //sequential output
      f_txn_g.users_count_exp <= (cs == STORE)? f_txn_g.users_count_exp+1 : f_txn_g.users_count_exp;
      cs = ns;
   end
   //Comb output
   f_txn_g.y_exp = (cs == STORE)? 1:0;
endtask

task assert_reset;
   //assert reset
   rst = 1;
   check_rst;
   //desert reset
   rst = 0;
endtask

task check_rst;
   @(negedge clk);
   if (users_count !== 0 && y !== 0) begin
     error_count = error_count + 1;
     $display("%t: Error: Reset value is asserted and the output is not tied low", $time);
   end
   else
     correct_count = correct_count + 1;
endtask
endmodule
```

Kareem Waseem

Q4.

```
module assoc_exercise;

bit [23:0] assoc[int];

initial begin

   assoc[0]     = 24'hA50400; // reset
   assoc[32'h400] = 24'h123456; // Instruction 1
   assoc[32'h401] = 24'h789ABC; // Instruction 2
   assoc[32'hF_FFFF] = 24'h0F1E2D; // Return from interrupt

   $display("The # of entries is %d", assoc.num);

   foreach (assoc[i]) begin
      $display("assoc[%h] = %h", i, assoc[i]);
   end

end
endmodule // test
```

Q5.

```
module struct_;

   typedef bit [6:0] bit7_t;

         typedef struct {
          bit7_t header;
          bit7_t cmd;
          bit7_t data;
          bit7_t crc;
          } packet;

   packet my_packet;

   initial begin
      my_packet.header = 7'h5A;
   end

endmodule
```

Kareem Waseem